Empirical Detection of Statistical and Differential Weaknesses in Hash Functions

A black-box toolkit, its calibration, and what it cannot see

John & Claude | R57 Labs | Version 2.0, October 2026 (v1.0: April 2026)

Abstract

We present **cryptid**, an open-source toolkit for black-box assessment of hash functions. It runs fourteen tests in four tiers: an aggregate statistical suite over input/output records with a logistic-regression classifier; a differential profile (empirical Strict Avalanche and Bit Independence Criteria) over chosen inputs; extended structural tests (near-collisions, sequence correlation, cycles); and an exhaustive low-weight linear-approximation search. A separate known-answer mode checks implementations of standard algorithms against a reference, because no statistical test can: SHA-256 with a corrupted round constant, or with 24 of its 64 rounds, passes the entire battery. We calibrate the classifier on four planted-weakness families and report its accuracy on variants generated from unseen seeds (100.0% accuracy, 0 false positives, 100.0% recall on 96 variants), then test generalization on five weakness types the detectors were not designed for, catching 4 of 5. Across 24 real functions at 10,000 samples, every cryptographic primitive is indistinguishable from a random oracle at the sensitivity tested, 7 of 7 weak non-cryptographic hashes are flagged, and the two well-designed ones (MurmurHash3, SipHash-2-4) are not. Jenkins one-at-a-time, whose weak final-byte avalanche is well documented, serves as a positive control: invisible to aggregate statistics, caught by the differential tier. White-box diffusion curves for six hashes, aligned on the round at which a flipped message word enters the state, put SHA-3 at full diffusion 3 rounds after absorption and SHA-256 at 90% 5 rounds after injection, MD5 at 8 and SHA-1 at 11; they also show that SM3's apparent early-round advantage in absolute round counts is a message-expansion effect, not faster mixing. This revision follows an independent review of version 1.0 that found the code correct and several of its claims wrong; Section 8 lists them.

Keywords: hash function testing, statistical testing, differential cryptanalysis, strict avalanche criterion, known-answer testing, diffusion analysis, AI-assisted development

1. Introduction

How do you know a hash function does what it claims? The question has two very different forms. The first is about the function: is its output statistically indistinguishable from random, and does it respond to input changes the way an ideal function would? The second is about an implementation: is this HSM's, this firmware's, this library's SHA-256 actually SHA-256? Version 1.0 of this paper blurred the two. They need different tools, and cryptid now provides both and says which is which.

The first question has practical weight. Hash-table hashes with poor avalanche cause clustering; checksums get used as if they were hashes; proprietary or embedded implementations arrive without source and without test vectors. History also shows that standards can be weakened deliberately: in 2013 leaked documents indicated that the NSA had engineered the Dual EC DRBG standard to contain a trapdoor, and the CIA and BND sold compromised Crypto AG cipher machines for decades. We note at the outset what statistical testing can and cannot say about such cases: Dual EC's output is statistically perfect, and no input/output test detects it. The threat model in Section 6 is explicit about the boundary.

We approach the first question empirically. The toolkit treats a hash function as a black box and applies a layered battery: aggregate statistics on input/output records, differential tests on chosen inputs, structural tests for sequence and cycle behaviour, and a linear-approximation search. No single test catches every kind of weakness; the battery is designed so that a weakness invisible to one tier is visible to another, and the real-function results in Section 4 are a demonstration of exactly that.

For the second question the toolkit provides a known-answer mode (*kat*) and an output comparison mode (*compare*). Section 3.4 shows why they are not optional: a SHA-256 with a one-bit typo in a round constant, or with its round count cut from 64 to 24, is still a well-mixed function and passes every statistical and differential test we run. Statistical quality and correctness are different properties.

The toolkit was built by a non-cryptographer working with AI assistance, as a deliberate test of that way of working. Version 1.0 was then reviewed adversarially by a second, independent model before publication. The review found the code correct and a number of the claims around it overstated or wrong. Section 8 lists them and what changed; the full review is in the repository.

2. Methodology

The toolkit runs fourteen tests in four tiers (Table 1). Tier 1 needs only input/output records and is the only tier that can run from a file; tiers 2–4 choose their own inputs and need a callable function — a built-in algorithm or an external command. In tiers 2–4 the function is fed raw random bytes, so a single-bit flip in the generated input is a single-bit flip in the hashed message. (Version 1.0 hex-encoded the bytes before hashing, so 44% of its "single-bit" flips changed 2–5 bits of the actual message.) In tier 1 the records' plaintexts are text and the hashed message is their UTF-8 encoding.

Tier	Tests	Needs	Asks
1 Statistical suite	bit correlation (every input × output bit) · output entropy · inter-record distance · conditional byte frequency · mutual information (permutation-tested) · multi-byte interaction (byte parities, pair/triple XOR and sum features × every output bit) + logistic-regression classifier	records	Does the output look random?
2 Differential profile	single-bit differential matrix (SAC) · byte-level differential · Hamming-distance profile at input distances 1, 2, 4, 8, 16 · differential bit independence (BIC) over all output-bit pairs	callable	Does the output change randomly?
3 Extended analysis	near-collision frequency · sequence correlation on counter-mode inputs · cycle and fixed-point detection (Floyd)	callable	Is there sequence or cycle structure?
4 Linear approximation	bias of parity(input mask) = parity(output mask) for all 1-, 2- and 3-bit input masks × every output bit, every input bit × all 2-bit output masks, sampled 2×2 masks, byte-parity masks (~25M approximations for a 256-bit digest)	callable	Is there a linear bias?

Table 1. The four tiers. Levels: *quick* = tier 1; *standard* = tiers 1–2; *full* = all four.

2.1 Statistical suite and classifier

Each tier-1 test produces a decision and a signal strength in [0, 1]. Bit correlation and multi-byte interaction are chi-squared tests on 2×2 tables; both now cover every input bit (80 for the generated 10-byte messages) and every output bit, which version 1.0 did not (it scanned input bits 0–31 and, for interactions, output bits 0–31 — a gap the held-out evaluation in Section 3.2 exposed). Conditional frequency tests whether output-byte distributions depend on input features (length parity, first-character high bit, byte-sum parity). Mutual information between the first input and output bytes is tested against 200 permutations at the 99th percentile. Entropy is measured per bit and per byte. The inter-record distance test measures the Hamming distance between digests of consecutive records; it was mislabelled "avalanche" in version 1.0, which it is not — the records are unrelated inputs — and it has been renamed.

Because thresholds depend on sample size, output width and the number of tests, a logistic-regression classifier combines 20 normalized features (signal strengths, chi-squared values divided by N, significant-pair fractions, phi coefficients, entropy minima, output width, log N) into one probability P(weakened) with a decision threshold learned on the training set (0.40). Logistic regression was chosen over tree ensembles for interpretability: the report names the features that drove a score, so a WARN is explainable even when no single test crossed its display threshold.

2.2 Differential profile

The single-bit differential matrix estimates P(output bit j flips | input bit i flips) for every (i, j) and tests each against 0.5 — the empirical Strict Avalanche Criterion. The bit-independence test packs each output bit's flip indicator across samples into a bitset and evaluates the phi coefficient for every pair of output bits (32,640 pairs for 256 bits) for eight sampled input bits; it also counts output bits whose flip indicator is constant across samples, which is a bit-independence failure phi cannot express (for a linear function such as CRC32, every column is constant). Byte-level differential measures the entropy of output-byte XOR deltas; the Hamming-distance profile checks the output distance distribution for input pairs at fixed distances.

2.3 Extended analysis and linear approximation

Near-collision frequency compares k-bit prefix matches for 1-bit-apart input pairs against random pairs. Sequence correlation hashes counter-mode inputs and tests lag-1 autocorrelation, consecutive Hamming distance and run lengths. Cycle detection iterates the function from random starting points with Floyd's algorithm. The linear-approximation tier, in the spirit of Matsui's linear cryptanalysis, evaluates the bias $|P(\text{parity}(\text{input mask}) = \text{parity}(\text{output mask})) - 1/2|$ with mask parities computed as big-integer bitset operations, which makes exhaustive coverage of all 1-, 2- and 3-bit input masks affordable; a signal requires the maximum bias to exceed $10/\sqrt{N}$ or the count of $p<0.001$ approximations to exceed five times its expectation. On a correct function the count of nominally significant approximations matches expectation to within a few percent, which is the check that the thresholds are calibrated.

2.4 Known-answer testing

kat runs an implementation — a command reading raw message bytes on stdin, or a file of its outputs — against the built-in reference for a named standard algorithm on published vectors (the empty message, "abc", the NIST 448- and 896-bit messages) and random messages at padding-boundary lengths (55, 56, 63, 64, 65, 111, 112, 127, 128, 135, 136 bytes and others), reporting the first differing digit of any mismatch. *compare* diffs two output files on their shared inputs and exits non-zero on any disagreement. The hand-rolled built-ins (SipHash-2-4, MurmurHash3, FNV-1a, Jenkins OAT, DJB2, CRC32, Adler-32) are themselves checked against published vectors on every run.

3. Calibration and validation

3.1 Label-blind synthetic calibration

A generator produces hash-function variants from seeded SHA-256: some clean, some with a planted weakness of one of four families at random severity — bit correlation (an output bit copies an input bit with probability p), frequency bias (an output byte's high bit follows an input feature), weak avalanche (leading output bytes replaced by a weak byte-sum hash), and subset leak (an output bit carries a parity or sum of a few input bytes). Variants are shuffled behind random codenames and the detectors run without labels. Version 1.0 called this "double-blind"; it is not. It is label-blind, and more importantly the four families were designed alongside the detectors and share their feature vocabulary, so accuracy on them measures calibration, not generalization. Sections 3.2–3.3 supply the missing evidence.

3.2 Held-out evaluation of the shipped classifier

The classifier (models/meta_learner_v4.json) is trained on 288 variants from 24 batches across four output widths (64, 128, 160, 256 bits) and three sample sizes (2,000, 10,000, 50,000). Two further sets of 96 variants each were generated from disjoint seed ranges (widths 64–256, N = 2,000 and 10,000). The first was a *development* set: evaluating the version-1.0 classifier on it is what exposed the coverage gaps described in Section 2.1 (every missed bit-correlation variant had its input bit outside the scanned window; most missed subset leaks had their output bit outside it), and evaluating the widened classifier on it exposed one more — leaks of the parity of a byte XOR, which the interaction features did not include. The features were extended and the classifier retrained. The second set is a *test* set used for no decision at all; Table 2 reports it. On the development set the final classifier scores 100.0% (0 false positives, 0 misses); on the untouched test set

100.0%, precision 1.000 (0 false positives among 42 clean variants), recall 1.000 (0 of 54 weakened variants missed). Clean variants score 0.04–0.15; weakened variants 0.47–1.00; the decision threshold is 0.40 (the F1-optimal training threshold was 0.25, but held-out results are identical from 0.25 to 0.50 and 0.40 keeps a margin above the scores of real primitives at small N). Version 1.0 quoted "93.3% on held-out data"; that figure was the training accuracy of an earlier model file, and no held-out evaluation existed.

Breakdown	n	Accuracy
weakness type: bit_correlation	23	100.0%
weakness type: clean	42	100.0%
weakness type: frequency_bias	10	100.0%
weakness type: subset_leak	12	100.0%
weakness type: weak_avalanche	9	100.0%
output width: 64 bits	24	100.0%
output width: 128 bits	24	100.0%
output width: 160 bits	24	100.0%
output width: 256 bits	24	100.0%
sample size: 2,000	48	100.0%
sample size: 10,000	48	100.0%
overall (test set)	**96**	**100.0%**

Table 2. Evaluation of the shipped classifier on 96 synthetic variants from unseen seeds (test set; threshold 0.40). TP=54, FP=0, TN=42, FN=0.

3.3 Out-of-family weaknesses

To test what the battery finds when the weakness was not designed to match a detector, five variants were written after the detectors were frozen (Table 3), plus a clean control, and the full battery was run on each. A *dead input byte* (the message's sixth byte is zeroed before hashing) has perfect output statistics; a *tail copy* sets the last digest byte to the last message byte with probability 0.7 — outside every range the v1.0 statistical suite scanned; *output redundancy* copies the first digest byte into the last with probability 0.7, a dependency between output bytes that no input feature predicts; a *permutation-invariant* hash sorts the message bytes before hashing, so every permutation of a message collides; and a *3-bit linear mask leak* sets one output bit to the XOR of three input bits in different bytes with probability 0.7, which is pairwise independent of every single input bit.

Variant	Statistical P	Differential	Extended	Linear	Verdict
clean control (sha256)	0.083	—	—	—	PASS
dead input byte	0.075	single bit, byte diff, hamming, bit independence	near collision	—	FAIL
tail copy	1.000	single bit, byte diff, hamming, bit independence	sequence	signal	FAIL
output redundancy	0.069	bit independence	—	—	WARN
permutation invariant	0.330	—	—	—	PASS
linear mask leak	1.000	single bit	—	signal	FAIL

Table 3. Out-of-family weaknesses, full level, N = 5,000. 4 of 5 are detected; the permutation-invariant hash is not, by any tier — a structural weakness with no statistical signature.

The pattern is the point of the layered design. The dead input byte and the output redundancy are invisible to the statistical suite (its tests look for dependence, and a dead byte produces independence; no input feature predicts a copied output byte) and are caught by the differential tier — the latter only because bit independence is now tested over every output-bit pair. The 3-bit linear leak is found twice: by the single-bit differential (each of its three input bits flips the leaked output bit with probability 0.85) and by the exhaustive 3-bit level of the linear tier, which reports the planted mask exactly. The tail copy is caught by every tier, including the statistical suite,

because a copied text byte has low entropy. The permutation-invariant hash passes everything (classifier score 0.33, elevated by the duplicate digests that anagram inputs produce but below threshold): its digests are uniform and avalanche perfectly, and second preimages are nevertheless trivial. That is the boundary of black-box statistical testing, stated by example.

3.4 Unfaithful implementations

Implementation	Statistical P	Differential	Verdict
sha256 correct (pure py)	0.152	—	PASS
sha256 K[20] 1-bit typo	0.154	—	PASS
sha256 24 of 64 rounds	0.149	—	PASS
sha256 12 of 64 rounds	0.176	—	PASS

Table 4. SHA-256 variants that are not SHA-256, standard level, N = 2,000. All pass. 24-step SHA-256 is below the best published practical collision attack (31 steps).

Every variant in Table 4 passes. A corrupted constant or a shortened round count leaves a function that is still a good pseudo-random mapping; nothing in its input/output behaviour says "not the standard". The only detector is disagreement with a reference on the same inputs, which is what *kat* and *compare* test, and why the toolkit no longer describes the statistical battery as a way to verify that an implementation is faithful.

4. Results on real functions

Table 5 is the detection matrix for the 24 built-in functions and their matched random-oracle controls at the full level with N = 10,000 samples (statistical suite: 10,000 records of mixed text inputs; differential and extended tiers: 10,000 chosen 10-byte inputs each; linear tier: 10,000 inputs). The block-cipher entries are constructions, not the primitives themselves: AES-128, SM4 and Camellia-128 as CBC-MAC with a fixed key, and ChaCha20 as a PRF whose 256-bit key is the padded message (version 1.0 keyed ChaCha20 with a fixed key and used nonce = MD5(message), which exercised MD5's diffusion rather than ChaCha20's).

Function	Type	Bits	Meta P	Differential	Extended	Linear	Verdict	Time
sha256	hash	256	0.06	—	—	—	PASS	88s
sha512	hash	512	0.10	—	—	—	PASS	113s
sha1	hash	160	0.05	—	—	—	PASS	66s
sha224	hash	224	0.07	—	—	—	PASS	79s
sha512_256	hash	256	0.07	—	—	—	PASS	101s
sha3_256	hash	256	0.06	—	—	—	PASS	100s
sha3_512	hash	512	0.10	—	—	—	PASS	111s
blake2b	hash	256	0.07	—	—	—	PASS	83s
blake2s	hash	256	0.07	—	—	—	PASS	78s
md5	hash	128	0.05	—	—	—	PASS	56s
sm3	hash	256	0.08	—	—	—	PASS	102s
aes128	cipher	128	0.05	—	—	—	PASS	261s
chacha20	cipher	256	0.06	—	—	—	PASS	282s
sm4	cipher	128	0.04	—	—	—	PASS	280s
camellia128	cipher	128	0.05	—	—	—	PASS	273s
crc32	checksum	32	0.04	SYHI	NQ	—	FAIL	32s
adler32	checksum	32	1.00	SYHI	NQC	S	FAIL	10s
fnv1a_32	non-crypto	32	1.00	SYHI	NQ	—	FAIL	55s
fnv1a_64	non-crypto	64	1.00	SYHI	NQ	—	FAIL	68s
murmur3_32	non-crypto	32	0.06	—	—	—	PASS	96s

Function	Type	Bits	Meta P	Differential	Extended	Linear	Verdict	Time
siphash24	PRF	64	0.05	—	—	—	PASS	349s
djb2	non-crypto	32	1.00	SYHI	NQC	—	FAIL	43s
jenkins	non-crypto	32	0.05	SI	Q	—	FAIL	71s
pearson64	non-crypto	64	0.06	S	—	—	WARN	102s
random_oracle_32	control	32	0.03	—	—	—	PASS	45s
random_oracle_64	control	64	0.04	—	—	—	PASS	51s
random_oracle_128	control	128	0.04	—	—	—	PASS	65s
random_oracle_160	control	160	0.04	—	—	—	PASS	74s
random_oracle_224	control	224	0.05	—	—	—	PASS	84s
random_oracle_256	control	256	0.05	—	—	—	PASS	94s
random_oracle_512	control	512	0.09	—	—	—	PASS	129s

Table 5. Detection matrix, full level, N = 10,000. Meta P = classifier P(weakened), threshold 0.40. Differential: S single-bit, Y byte, H Hamming profile, I bit independence. Extended: N near-collision, Q sequence correlation, C cycles. Linear: S = bias above threshold. Time is wall-clock for all four tiers on a 2-vCPU container.

4.1 Cryptographic primitives

All 15 cryptographic functions pass every test, with classifier probabilities 0.04–0.10 against 0.03–0.09 for the random-oracle controls. We state this result at its actual size: at N = 10,000 and the sensitivity of these tests, no statistical bias, differential non-uniformity, sequence correlation or low-weight linear bias is detectable. It is a null result, not a certificate; Section 3.4 shows what else passes.

4.2 Non-cryptographic functions

The 9 non-cryptographic functions divide as expected. 7 of the 7 designed without regard to avalanche or linearity are flagged (crc32, adler32, fnv1a_32, fnv1a_64, djb2, jenkins, pearson64); of these, 4 are flagged by the statistical suite alone (adler32, fnv1a_32, fnv1a_64, djb2) and the rest only by the differential and structural tiers. MurmurHash3 and SipHash-2-4 pass everything. MurmurHash3 makes no cryptographic claim but was designed for avalanche quality; SipHash is a keyed cryptographic PRF (version 1.0 wrongly said neither claims cryptographic security). Their clean results confirm that statistical quality is a design choice rather than a by-product of complexity.

4.3 Jenkins one-at-a-time as positive control

Jenkins OAT passes the aggregate suite (P = 0.05) and fails the differential tier: the single-bit matrix shows input bits in the final message byte flipping some output bits far from half the time, and bit independence fails. This is a known property, not a finding of ours: Wikipedia's article on the function states that "only a few bits in the last byte of the input key are weakly mixed to a minority of bits in the output hash", and SMHasher lists one-at-a-time as failing its avalanche and BIC tests. Version 1.0 called it "previously uncharacterized" and attributed it to a missing final mixing pass; the function has a final mixing pass (three operations after the byte loop, present in our own implementation), and the cause is that a difference in the last byte passes through only one per-byte mix and that short finalizer. Its value here is as a positive control — a weakness that aggregate statistics cannot see and differential tests can — which is also how we use CRC32, whose XOR-linearity makes every single-bit differential deterministic (every flip column constant) while its output distribution is uniform (P = 0.04).

5. Internal state diffusion (white-box)

For six hashes with source available we instrument pure-Python implementations (verified against hashlib) to record the working state after every compression round, flip one message bit, and measure the Hamming distance between the two state trajectories. Version 1.0 reported the normalized distance at absolute round numbers and compared designs on it. That comparison was confounded: a Merkle–Damgård compression function consumes one message word per round for its first 16 rounds, so a flip in word k cannot touch the state

before round k (round k–4 in SM3, whose expansion folds W[j+4] into round j), whereas the Keccak sponge absorbs the whole block before round 1. At "round 5" of SHA-256 most of a 55-byte message had not entered the state at all. Table 6 reports both the absolute figures and the figures aligned on the round at which the flipped word enters the state; only the aligned ones compare mixing speed.

Algorithm	State	Rounds	Absolute: rounds to 50% / 90%	Aligned: rounds to 50% / 90%	Aligned diffusion at +3 / +5
SHA-3-256 (sponge)	1600 b	24	3 / 3	3 / 3	99.9% / 100.0%
SM3	256 b	64	7 / 12	4 / 6	31.0% / 77.2%
SHA-256	256 b	64	10 / 16	4 / 5	43.4% / 90.5%
SHA-512	512 b	80	11 / 16	4 / 6	29.1% / 76.2%
MD5	128 b	64	12 / 18	5 / 8	18.6% / 52.0%
SHA-1	160 b	80	14 / 20	7 / 11	9.2% / 26.3%

Table 6. Internal diffusion, 2,000 random 1-bit flips per algorithm, normalized to half the state width. "Absolute" counts from the start of the compression function and depends on when the flipped word is absorbed; "aligned" counts from the round the flipped word enters the state.

Aligned, the picture is simpler and smaller than version 1.0 made it. Keccak-f reaches full diffusion 3 rounds after absorption, consistent with its designers' analysis. SHA-256 reaches 90% 5 rounds after a word enters (90.5% at +5), not 16. SM3's absolute lead over SHA-256 in version 1.0 ("nearly double") was entirely the four-round head start its message expansion gives each word; aligned, SM3 is at 77.2% at +5 against SHA-256's 90.5%, and the claim that its designers accelerated the round function is withdrawn. MD5 (8 rounds to 90%) and SHA-1 (11) do mix more slowly per round than SHA-256, which is consistent with but does not explain their collision attacks: those exploit specific message-schedule and round-function algebra, and MD5 fell thirteen years before SHA-1 despite mixing faster. Diffusion rate is a descriptive property of a design, not a security metric, and we draw no security ranking from Table 6.

6. Limitations and threat model

Weakness class	Example	Detectable by cryptid?	Requires
Statistical bias in output	calibration families; tail copy	Yes (tier 1)	input/output records
Differential / avalanche weakness	Jenkins OAT, FNV-1a, dead input byte	Yes (tier 2)	chosen inputs
Deterministic differentials, linearity	CRC32, Adler-32	Yes (tiers 2–4)	chosen inputs
Low-weight linear bias	3-bit mask leak	Yes (tier 4), up to 3 input bits	chosen inputs
Output-bit dependency	output redundancy	Yes (tier 2, BIC over all pairs)	chosen inputs
Unfaithful but well-mixed implementation	wrong constant, reduced rounds	No — use kat / compare	a trusted reference
Structural, no statistical signature	permutation-invariant hash	No	cryptanalysis / source
Kleptographic trapdoor	Dual EC DRBG	No	parameter analysis
Reduced keyspace	DES, A5/1	No	specification review
Collision / preimage resistance	round-reduced SHA-256	No	cryptanalysis

Table 7. What the toolkit does and does not detect.

Sensitivity is bounded by the sample count: a bias too small to reach the chi-squared thresholds at the chosen N is invisible, and the thresholds are deliberately conservative so that real primitives never trip them. The statistical suite's features are defined on text records of the generator's distribution; a file of records from a very different input distribution is still valid input, but the classifier's calibration was not measured on it. Tiers 2–4 require the function to be callable on arbitrary byte strings. Timings (Table 8) are for a slow container; the pure-Python implementation trades speed for having no dependencies.

Level	Tiers	Jenkins OAT (32-bit), N=5,000	SHA-256 (256-bit), N=5,000	SHA-256, N=10,000
quick	1	1 s	2 s	4 s
standard	1–2	3 s	7 s	14 s
full	1–4	42 s	49 s	85 s

Table 8. Wall-clock time per level on a 2-vCPU Linux container (about 1.5× slower than a current laptop). Times scale with N and with output width.

7. Related work

SMHasher (Appleby; maintained fork by Urban) is the standard quality suite for non-cryptographic hash functions and includes avalanche, bit-independence, differential, sparse-key and distribution tests; it requires the function to be compiled in as C/C++, and its published results already record Jenkins one-at-a-time's failures. cryptid trades SMHasher's depth for black-box access — anything that returns a digest can be tested, including hardware — and adds the record-based statistical screening, the classifier, and the known-answer workflow for implementations. The NIST Statistical Test Suite (SP 800-22), Dieharder and TestU01 evaluate the randomness of a bit stream rather than an input→output relationship. The Strict Avalanche and Bit Independence Criteria are due to Webster and Tavares (1985); our tier 2 is an empirical evaluation of both. The linear tier is a black-box descendant of Matsui's linear cryptanalysis (1993), and the differential tier of Biham and Shamir's differential cryptanalysis (1990), applied without knowledge of the algorithm. The Keccak team's own documentation describes the permutation's diffusion, with which Table 6 agrees.

8. Revision history: what the review found

Version 1.0 of this paper, the toolkit and its website were reviewed before announcement by a model from a different family than the one that helped build them, with the instruction to find what was wrong. The review verified every hand-rolled hash against published vectors, reproduced the headline numbers, and ran experiments where a claim looked weak. It found the code correct and the following claims not:

• Jenkins OAT's weakness was called "previously uncharacterized" and attributed to a missing final mixing pass. It is documented (Wikipedia, SMHasher) and the function has a finalizer. Reframed as a positive control (§4.3).

• The diffusion comparison across sponge and Merkle–Damgård designs was confounded by message-word injection timing; the SM3 "deliberate design choice" claim and a misuse of "security margin" rested on it. Re-measured with injection alignment (§5); the SM3 claim reverses.

• Calibration was called "double-blind"; the planted families mirrored the detectors' features; the quoted "93.3% held-out" accuracy was an older model's training accuracy. Renamed, and replaced by a real held-out evaluation (§3.2) and an out-of-family test (§3.3).

• SipHash was said to make no cryptographic claim. It is a cryptographic PRF (§4.2).

• The toolkit was framed as a way to verify that an implementation is faithful. It cannot be; a known-answer mode was added and the limitation is demonstrated (§3.4).

• The statistical suite's "avalanche" test compared unrelated records; the chosen-input tiers hashed hex text rather than the generated bytes; bit correlation, interaction and linear approximation covered only windows of the input and output. All corrected (§2), and the classifier retrained on the widened features.

• Counts of methods, several table values, and the "raw results available" statement were inconsistent or untrue. Every number here is now generated from the JSON files in results/, and the paper is built from them.

• The repository claimed an MIT license without a LICENSE file; the site's hardware wrapper examples did not run; a file with inconsistently padded digests produced a spurious warning; a phase that crashed could still report PASS. All fixed.

The pattern is worth stating because it is the result of the experiment this project set out to run. The errors were in the narrative layer — domain knowledge, literature, interpretation, presentation — not in the mechanical one. The code the first model wrote was right; the story told about it was not checked against the code, the literature, or an adversary. A second independent pass, asked to break the work rather than confirm it, found the problems in an afternoon. We take the lesson to be procedural: generation and verification are different jobs, and the verification job needs a different agent and a different instruction. The complete review and change list are in the

repository (docs/REVIEW.md).

9. Conclusion and future work

A layered black-box battery with a conservative classifier on top separates hash functions with statistical or differential weaknesses from those without, at the sample sizes an engineer can afford to run, with no false positives on real primitives in anything we have tried. Its reach is bounded, and we have tried to state the bound by example rather than by adjective: it does not see unfaithful-but-well-mixed implementations, structural weaknesses without a statistical signature, trapdoors, or anything cryptanalytic. For the first of these the toolkit now carries the right tool. Future work includes algebraic-degree testing, a wider exhaustive mask range in the linear tier (4-bit input masks are within reach with the bitset approach), calibration on additional input distributions, and a reference corpus of real-world hash implementations with known defects.

Source code, models, experiment scripts, the JSON results behind every table, and the v1.0 review are at github.com/r57-labs/cryptid. Contact: mail@r57labs.com